

Relatório Arquitetural do Projeto Jackut

Sumário

1. Estrutura arquitetural adotada
2. Padrão Arquitetural em Camadas
3. Padrão MVC adaptado
4. Padrão Facade
5. Padrão Controller
6. Padrão Service Layer
7. Padrão Domain Model
8. Padrão Repository
9. Abstract Repository com Generics
10. Padrão Singleton
11. Padrão State no módulo de amizades
12. Padrão Identity Field
13. Índices auxiliares como estratégia de acesso
14. Padrão Queue/FIFO nas mensagens
15. Separação entre Message e ChatMessenger
16. Persistência por serialização XML
17. Tratamento de exceções de domínio
18. Relação entre os padrões adotados
19. Síntese arquitetural

1. Estrutura arquitetural adotada

O projeto foi estruturado com base em uma combinação de padrões arquiteturais e padrões de projeto voltados à separação de responsabilidades, organização do domínio e isolamento da persistência.

Os principais padrões e estratégias empregados são:

- Arquitetura em Camadas;
- MVC adaptado para aplicação sem interface gráfica;
- Facade;
- Controller;
- Service Layer;

- Domain Model;
- Repository;
- Abstract Repository com Generics;
- Singleton;
- State;
- Identity Field;
- filas FIFO para ordenação de mensagens;
- exceções de domínio.

A escolha por combinar esses padrões tem como objetivo manter o código organizado em módulos com responsabilidades distintas. A aplicação possui uma interface pública simples, mas internamente distribui comportamento entre controladores, serviços, entidades e repositórios.

Essa decisão evita que a lógica de domínio fique concentrada na **Facade** ou diretamente nos repositórios. A **Facade** recebe chamadas externas, os controladores coordenam casos de uso, os serviços aplicam regras de negócio, as entidades representam o estado do domínio e os repositórios cuidam da persistência.

2. Padrão Arquitetural em Camadas

O padrão arquitetural predominante no projeto é a Arquitetura em Camadas.

Nesse estilo, o sistema é dividido em níveis de responsabilidade. Cada camada possui uma função específica e se comunica preferencialmente com a camada imediatamente abaixo ou com componentes bem definidos.

No projeto, a divisão pode ser representada da seguinte forma:

- camada de entrada: **Facade**;
- camada de orquestração: **controllers**;
- camada de regras de negócio: **services**;
- camada de domínio: **models**;
- camada de persistência: **repositories**;
- camada de infraestrutura de persistência: **XMLController**.

A camada de entrada oferece os métodos públicos utilizados pelos testes. A camada de controladores organiza os casos de uso. A camada de serviços concentra validações e regras. A camada de domínio representa os conceitos principais por meio de entidades. A camada de persistência encapsula armazenamento, busca e recuperação de dados.

O uso dessa arquitetura foi escolhido porque o projeto possui várias responsabilidades diferentes. Se todas elas fossem implementadas em uma única classe, haveria alto acoplamento, baixa coesão e dificuldade de manutenção. A separação em camadas permite localizar rapidamente onde cada tipo de decisão deve ser implementado.

Outro benefício importante é a possibilidade de evolução. Uma mudança na persistência não precisa alterar diretamente as entidades. Uma mudança nas regras de negócio não precisa alterar a **Facade**. Uma mudança nos comandos externos tende a ficar restrita à entrada e aos controladores.

3. Padrão MVC adaptado

O projeto utiliza uma adaptação do padrão Model-View-Controller.

O MVC tradicional é composto por:

- Model: domínio e regras;
- View: apresentação;
- Controller: interpretação da entrada e coordenação da resposta.

No Jackut, não há uma camada de visão gráfica. A interação ocorre por chamadas de métodos feitas pelos testes de aceitação. Por isso, a arquitetura se aproxima de um MVC adaptado, no qual a **Facade** cumpre o papel de fronteira de entrada e saída.

Nesse contexto:

- o Model é representado por entidades, serviços e repositórios;
- o Controller é representado por `UserController` e `ChatMessengerController`;
- a View tradicional não existe, sendo substituída pela interface textual esperada pelos testes.

O MVC foi usado como referência porque permite separar a entrada do sistema da lógica de negócio. Mesmo sem interface gráfica, ainda existe a necessidade de separar quem recebe comandos de quem executa regras de domínio.

Essa adaptação também favorece a testabilidade. Os testes chamam a **Facade**, mas a lógica real está distribuída em classes especializadas. Assim, o sistema mantém uma fronteira pública simples sem abrir mão de organização interna.

4. Padrão Facade

O padrão Facade é utilizado diretamente pela classe **Facade**.

Esse padrão fornece uma interface simplificada para um conjunto mais complexo de subsistemas. Em vez de expor controladores, serviços, repositórios e entidades diretamente para quem utiliza a aplicação, a **Facade** oferece métodos de alto nível.

No projeto, a **Facade** concentra os métodos exigidos pelos testes, como:

- criação de usuário;
- abertura de sessão;
- edição de perfil;
- operações de amizade;
- envio e leitura de recados;

- encerramento e reinicialização do sistema.

A justificativa para esse padrão é bastante clara: os testes de aceitação precisam interagir com uma API simples e estável. Internamente, cada operação pode envolver vários objetos, mas externamente ela aparece como uma chamada única.

Por exemplo, uma operação pública pode exigir validação, consulta de usuário, criação de entidade, atualização de índices e persistência posterior. A **Facade** oculta essa complexidade de quem chama o sistema.

O padrão também reduz acoplamento externo. Caso a implementação interna mude, os scripts de teste não precisam conhecer a nova organização, desde que os métodos públicos permaneçam compatíveis.

5. Padrão Controller

Os controladores aparecem nas classes:

- `UserController`;
- `ChatMessengerController`.

O padrão Controller é usado para coordenar casos de uso. Ele não deve representar o domínio em si, nem deve ser responsável por persistência direta. Sua função é receber uma solicitação da camada de entrada e organizar a chamada aos serviços necessários.

No projeto, o controlador tem papel especialmente importante porque algumas operações afetam mais de uma entidade. A criação de um usuário, por exemplo, envolve estruturas diferentes. O envio de uma mensagem também envolve chat, mensagem, caixa de notificação e estado de leitura.

Sem controladores, a **Facade** precisaria conhecer todos esses detalhes de composição. Isso deixaria a classe pública muito grande e misturaria entrada com regra de aplicação.

O Controller foi utilizado, portanto, para:

- reduzir a complexidade da **Facade**;
- coordenar serviços em operações compostas;
- transformar dados de entrada quando necessário;
- manter a lógica de caso de uso separada da lógica interna de cada serviço.

6. Padrão Service Layer

O padrão Service Layer é representado pelo pacote `services`.

Esse padrão cria uma camada intermediária responsável pelas regras de negócio da aplicação. Os serviços não são simples repositórios e também não são apenas entidades. Eles representam operações relevantes para o domínio.

No projeto, os principais serviços são:

- `UserService`;

- `ProfileService`;
- `FriendshipService`;
- `MessageBoxService`;
- `ChatMessengerService`;
- `MessageService`;
- `UserIntegrator`.

O uso de Service Layer se justifica porque muitas regras não pertencem naturalmente a uma única entidade. Validar criação de usuário, aceitar amizade, localizar destinatário e enviar mensagem são ações que envolvem coordenação de objetos e repositórios.

A camada de serviços impede que essas regras fiquem dentro dos controladores. O controlador coordena o caso de uso; o serviço executa a regra de negócio.

Essa separação melhora a coesão do projeto. Por exemplo, regras de usuário ficam em `UserService`, regras de perfil em `ProfileService`, regras de amizade em `FriendshipService` e regras de mensagens em `MessageService` e `ChatMessengerService`.

Além disso, a Service Layer facilita alterações futuras. Caso uma regra de validação mude, ela pode ser ajustada no serviço correspondente sem alterar a **Facade** nem os repositórios.

7. Padrão Domain Model

O padrão Domain Model aparece na representação das entidades do sistema.

Em vez de trabalhar apenas com estruturas genéricas de dados, o projeto define classes que representam conceitos do domínio:

- `User`;
- `Profile`;
- `Friendship`;
- `MessengerBox`;
- `ChatMessenger`;
- `ChatParticipantsKey`;
- `ChatUserState`;
- `Message`.

Cada uma dessas entidades encapsula parte do estado e algumas operações diretamente relacionadas a esse estado. Por exemplo, `Profile` sabe adicionar e consultar atributos; `Friendship` sabe adicionar e remover usuários de listas por estado; `ChatUserState` sabe mover uma mensagem de não lida para lida.

Esse padrão foi escolhido porque o sistema possui conceitos próprios e regras específicas.

Representar tudo como mapas soltos ou listas genéricas tornaria o código mais frágil e menos expressivo.

O Domain Model permite que o código reflita a linguagem do problema. Classes como `Message`, `Friendship` e `ChatMessenger` indicam diretamente quais conceitos existem na aplicação.

Outra vantagem é que o comportamento fica próximo do estado que manipula. O método que move uma mensagem para a fila de lidas está em `ChatUserState`, que é justamente a classe que possui as filas de leitura. Isso evita que serviços externos precisem manipular diretamente detalhes internos das entidades.

8. Padrão Repository

O padrão Repository é usado para isolar o acesso aos dados.

No projeto, os repositórios ficam divididos principalmente em dois grupos:

- repositórios de usuário;
- repositórios de chat e mensagens.

Exemplos:

- `UserRepository`;
- `ProfileRepository`;
- `FriendshipRepository`;
- `MessengerBoxRepository`;
- `ChatMessengerRepository`;
- `MessageRepository`.

O Repository fornece uma interface orientada ao domínio para armazenar e recuperar objetos. A camada de serviços não precisa saber como os dados são salvos, qual arquivo XML é utilizado ou como o mapa interno é carregado. Ela apenas solicita operações como salvar usuário, buscar perfil ou recuperar mensagem.

Esse padrão foi utilizado para evitar que regras de negócio dependam diretamente da infraestrutura de persistência. Se futuramente a persistência em XML fosse substituída por banco de dados, os serviços não deveriam precisar ser reescritos por completo. O impacto principal ficaria concentrado nos repositórios.

Outro motivo para usar repositórios é a organização dos índices auxiliares. Algumas buscas não ocorrem pelo ID principal da entidade. Por exemplo, usuários podem ser buscados por nome de usuário. Perfis e caixas de mensagem podem ser buscados pelo ID do usuário. Essas estratégias de busca são responsabilidade natural dos repositórios.

9. Abstract Repository com Generics

O projeto utiliza uma classe abstrata genérica chamada `AbstractRepository<T>`.

Essa estrutura reúne comportamentos comuns a todos os repositórios:

- manutenção de um mapa principal de entidades;
- inserção de objeto por ID;
- busca por ID;
- remoção;
- salvamento;
- reset dos dados.

O uso de Generics permite que a mesma implementação seja reaproveitada para diferentes tipos de entidade. Assim, `AbstractRepository<User>`, `AbstractRepository<Profile>`, `AbstractRepository<Message>` e outros repositórios compartilham uma base comum.

Essa solução está relacionada ao princípio DRY, isto é, evitar repetição de código. Sem essa abstração, cada repositório teria que repetir a lógica de mapa, salvamento e carregamento.

Também há uma relação com o padrão Template Method em sentido amplo. A classe abstrata define parte do comportamento geral, enquanto as subclasses especializam o que for necessário, como nome do arquivo, índices auxiliares e métodos de busca específicos.

O resultado é uma camada de persistência mais padronizada. Todos os repositórios seguem o mesmo ciclo básico: carregar dados, manter mapa em memória, salvar dados e limpar dados quando necessário.

10. Padrão Singleton

O padrão Singleton é utilizado em classes que devem possuir uma instância única durante a execução.

Ele aparece em:

- `XMLController`;
- repositórios;
- alguns serviços auxiliares, como `MessageBoxService` e `UserIntegrator`.

O Singleton garante que a aplicação utilize a mesma instância de determinado componente. No caso dos repositórios, isso é importante porque cada repositório mantém um mapa em memória. Se várias instâncias independentes fossem criadas, poderiam existir versões diferentes do mesmo conjunto de dados.

No caso do `XMLController`, o Singleton centraliza o acesso à lógica de arquivos. Isso evita espalhar criação de diretórios, montagem de caminhos e manipulação de XML por várias classes.

O padrão foi escolhido por simplicidade e coerência com o escopo do projeto. Como a aplicação é executada em um único processo e os testes interagem com uma instância da *Facade*, manter repositórios compartilhados é uma solução prática.

Entretanto, é importante observar que Singleton aumenta acoplamento global. Em sistemas maiores, seria comum utilizar injeção de dependência. Para o contexto acadêmico do Jackut, o Singleton oferece uma solução simples e suficiente.

11. Padrão State no módulo de amizades

O padrão State é aplicado no controle de amizades.

As classes relacionadas ficam no pacote `friendshipStateHandles`:

- `FriendShipStateHandle`;
- `NoFriendshipState`;
- `SentRequestState`;
- `ReceivedRequestState`;
- `CurrentFriendState`.

O objetivo do State é alterar o comportamento de uma operação de acordo com o estado atual do objeto ou da relação.

No projeto, a operação de adicionar amizade não possui sempre o mesmo comportamento. Ela depende da situação atual entre dois usuários:

- se não existe relação, deve criar uma solicitação;
- se já existe solicitação enviada, deve impedir duplicidade;
- se existe solicitação recebida, deve aceitar a amizade;
- se já são amigos, deve impedir nova adição.

Sem o State, essa lógica provavelmente ficaria em um único método com várias condições. Com o padrão, cada estado possui uma classe responsável por sua própria regra.

Esse desenho melhora legibilidade e manutenção. Quando uma regra de transição precisa mudar, ela pode ser alterada na classe de estado correspondente, sem transformar o serviço de amizade em um bloco grande de condicionais.

O uso do State também torna explícito que amizade não é apenas uma lista, mas uma relação com ciclo de vida: inexistente, solicitada, recebida e confirmada.

12. Padrão Identity Field

O padrão Identity Field consiste em atribuir um identificador único para cada entidade relevante do domínio.

No projeto, as entidades utilizam IDs gerados, em geral, por UUID. Esse padrão aparece em entidades como:

- `User`;
- `Profile`;
- `Friendship`;
- `MessengerBox`;
- `ChatMessenger`;

- `Message`.

O uso de IDs internos separa identidade técnica de atributos visíveis. O nome de usuário, por exemplo, é importante para entrada e consulta, mas o relacionamento interno entre entidades fica mais estável quando feito por ID.

Essa decisão é especialmente útil porque várias entidades se relacionam:

- um perfil pertence a um usuário;
- uma estrutura de amizade pertence a um usuário;
- uma caixa de mensagens pertence a um usuário;
- uma mensagem pertence a um chat;
- um chat possui participantes.

Com Identity Field, essas relações podem ser representadas por identificadores, reduzindo dependência direta entre objetos complexos e facilitando a persistência em mapas.

13. Índices auxiliares como estratégia de acesso

Além do mapa principal herdado de `AbstractRepository<T>`, vários repositórios mantêm mapas auxiliares.

Esses índices funcionam como estruturas de acesso rápido para consultas que não usam diretamente o ID principal da entidade.

Exemplos:

- `UserRepository` mantém `userName` -> `userId`;
- `ProfileRepository` mantém `userId` -> `profileId`;
- `FriendshipRepository` mantém `userId` -> `friendshipId`;
- `MessengerBoxRepository` mantém `userId` -> `messengerBoxId`;
- `ChatMessengerRepository` mantém `ChatParticipantsKey` -> `chatId`.

Essa estratégia se aproxima de um padrão de indexação em memória. O objetivo é evitar buscas lineares em todos os objetos sempre que for necessário localizar uma entidade por uma chave alternativa.

O uso desses índices melhora a eficiência e simplifica a camada de serviços. O serviço pede uma entidade por uma chave de domínio e o repositório resolve como encontrá-la.

Outro ponto importante é que esses índices são reconstruídos quando os dados são carregados dos arquivos XML. Assim, a persistência armazena os objetos principais, e os repositórios reconstroem as estruturas auxiliares em memória no início da execução.

14. Padrão Queue/FIFO nas mensagens

O sistema utiliza filas para representar ordem de leitura e ordem de notificação.

Esse comportamento aparece principalmente em:

- `MessengerBox`;
- `ChatUserState`.

O padrão de fila segue a regra FIFO, isto é, First In, First Out. A primeira mensagem a entrar deve ser a primeira a sair.

Essa decisão é adequada porque recados devem ser lidos na ordem em que foram recebidos. Se uma mensagem mais recente fosse lida antes de uma antiga, o comportamento poderia violar a expectativa natural do usuário e dos testes.

Em `MessengerBox`, a fila representa notificações pendentes. Em `ChatUserState`, a fila representa mensagens não lidas e mensagens já lidas. A leitura remove a mensagem da fila de não lidas e a move para a fila de lidas.

Essa separação permite preservar a ordem e também manter histórico de leitura dentro do estado do participante.

15. Separação entre `Message` e `ChatMessenger`

Uma decisão arquitetural importante é a separação entre `Message` e `ChatMessenger`.

O `ChatMessenger` representa a conversa e o estado de leitura dos participantes. A entidade `Message` representa o conteúdo textual e a identidade da mensagem.

Essa separação evita que o conteúdo da mensagem seja duplicado em várias filas. Em vez de armazenar o texto completo em cada participante, o sistema armazena IDs de mensagens nas filas e mantém o conteúdo em uma entidade própria.

Essa escolha aproxima o projeto de uma arquitetura mais escalável. Em uma conversa com múltiplos usuários, uma única mensagem pode ser referenciada por vários participantes. Cada participante possui seu estado de leitura, mas todos apontam para a mesma entidade `Message`.

Essa decisão também permite evolução futura. A mensagem pode receber novos atributos sem alterar diretamente o chat:

- remetente;
- data de envio;
- horário;
- status;
- anexos;
- edição;
- remoção lógica.

Portanto, a separação entre mensagem e conversa reduz acoplamento, evita duplicação e prepara o domínio para funcionalidades mais complexas.

16. Persistência por serialização XML

A persistência utiliza serialização XML por meio de `XMLEncoder` e `XMLDecoder`.

O padrão adotado não é um banco de dados relacional, mas uma persistência simples de objetos serializados em arquivos. Cada repositório define o arquivo correspondente à sua entidade.

O componente central dessa estratégia é o `XMLController`, responsável por:

- criar a pasta de dados;
- montar caminhos de arquivos;
- salvar mapas em XML;
- carregar mapas de XML;
- remover arquivos no reset.

A decisão por XML é coerente com o contexto do projeto porque evita dependências externas. Não é necessário configurar banco de dados, servidor ou biblioteca adicional de persistência.

Como o projeto é validado por testes de aceitação locais, a persistência em arquivo é suficiente para verificar se o estado sobrevive entre execuções.

Ao mesmo tempo, o isolamento da persistência em repositórios diminui o custo de uma possível troca futura. A camada de serviços não depende diretamente de `XMLEncoder` ou `XMLDecoder`; ela depende dos repositórios.

17. Tratamento de exceções de domínio

O projeto utiliza exceções específicas para representar falhas conhecidas do domínio.

Exemplos:

- `LoginInvalido`;
- `SenhaInvalida`;
- `ContaComEsseNomeJaExiste`;
- `LoginOuSenhaInvalidos`;
- `UsuarioNaoCadastrado`;
- `AtributoNaoPreenchido`;
- `AdicionarASiMesmoAmigo`;
- `UsuarioJaAdicionadoAmigo`;
- `EsperandoAceitacaoAmigo`;
- `EnviarRecadoParaSiMesmo`;
- `NaoHaRecados`;
- `FileError`;

- `SaveError`.

Essa abordagem é preferível ao uso de exceções genéricas porque cada erro possui significado próprio no domínio. Uma senha inválida não é o mesmo problema que usuário inexistente; ausência de recados não é o mesmo que erro de salvamento.

As exceções também cumprem papel importante na integração com os testes. O `EasyAccept` valida mensagens de erro específicas, e as exceções customizadas permitem associar cada regra violada à mensagem esperada.

Do ponto de vista arquitetural, as exceções funcionam como parte do contrato do domínio. Elas documentam quais falhas são esperadas e quais situações foram tratadas explicitamente.

18. Relação entre os padrões adotados

Os padrões adotados não aparecem isoladamente. Eles se complementam.

A Arquitetura em Camadas define a organização geral. O MVC adaptado orienta a separação entre entrada, controle e modelo. A `Facade` fornece uma interface pública simples. Os `Controllers` coordenam casos de uso. A `Service Layer` concentra regras de negócio. O `Domain Model` representa os conceitos centrais. Os `Repositories` isolam persistência. O `Singleton` garante instâncias compartilhadas para componentes que mantêm estado em memória. O `State` organiza as transições de amizade.

Essa combinação permite que o sistema tenha uma interface simples sem abrir mão de uma organização interna mais elaborada.

Um exemplo dessa interação ocorre no envio de mensagem:

- a `Facade` recebe a chamada;
- o `controller` coordena o caso de uso;
- os serviços aplicam as regras;
- as entidades atualizam estado;
- os repositórios mantêm os objetos;
- a persistência salva posteriormente o estado em XML.

Esse encadeamento mostra como os padrões se distribuem em responsabilidades complementares.

19. Síntese arquitetural

A arquitetura do Jackut foi construída com foco em separação de responsabilidades, clareza estrutural e possibilidade de evolução.

Os padrões mais relevantes são:

- `Facade`, para simplificar o acesso externo;
- `Controller`, para organizar casos de uso;
- `Service Layer`, para concentrar regras de negócio;

- Domain Model, para representar conceitos do domínio;
- Repository, para isolar persistência;
- Abstract Repository com Generics, para reduzir repetição;
- Singleton, para compartilhar repositórios e infraestrutura;
- State, para representar transições de amizade;
- Identity Field, para relacionar entidades por identificadores;
- Queue/FIFO, para preservar ordem de mensagens.

A parte de chat e mensagens merece destaque porque foi modelada de forma mais extensível do que uma simples lista de recados. A separação entre `ChatMessenger`, `ChatUserState`, `ChatParticipantsKey`, `MessengerBox` e `Message` permite que o sistema trate conversas, participantes, notificações, estado de leitura e conteúdo como conceitos distintos.

Essa decisão cria uma base adequada para evolução futura para chats com múltiplos usuários. O uso de conjunto de participantes, estado individual por usuário e mensagens referenciadas por ID permite que uma mesma mensagem seja entregue a vários participantes sem duplicação de conteúdo e sem perder controle individual de leitura.

Portanto, a solução arquitetural adotada é compatível com o porte acadêmico do projeto, mas já apresenta organização modular suficiente para manutenção e extensão.